

ESTUDIO — Glosario de conceptos técnicos

Material de estudio para el final del TFC Bajoneá. Pensado para las **preguntas conceptuales y teóricas** de la mesa — las que no son sobre un archivo puntual sino sobre "¿qué es un DTO?" o "¿qué es un ORM?".

Cada término tiene: la definición en criollo, y **cómo se usa concretamente en Bajoneá**.

ARQUITECTURA Y CAPAS

API REST

Qué es: una forma de que dos programas se hablen por HTTP. El cliente pide algo con una URL y un verbo (GET, POST...), y el servidor responde con datos, normalmente en JSON.

Los principios que aplica Bajoneá:

- **Recursos identificados por URL:** `/api/v1/productos/5` es ese producto.
- **Verbos con significado:** GET lee, POST crea, PUT reemplaza, PATCH modifica parcialmente, DELETE borra.
- **Sin estado en el servidor:** cada request lleva toda la información necesaria (el token va en el header).

En Bajoneá: 16 controllers, todos con `@RestController` y rutas bajo `/api/v1/`.

Detalle del versionado: el `/v1/` en la ruta permite que en el futuro exista un `/v2/` con cambios incompatibles, sin romper a los clientes viejos.

Arquitectura en capas

Qué es: organizar el código en niveles, donde cada uno tiene una única responsabilidad y solo habla con el de al lado.

Las capas de Bajoneá:

Controller (HTTP) → Service (negocio) → Repository (acceso) → Base de datos

Por qué se hace así: si todo estuviera junto, cambiar la base de datos te obligaría a tocar la lógica de negocio; cambiar de HTTP a otra cosa te obligaría a reescribir todo. Separado, cada cambio queda contenido en su capa.

(Está explicado en detalle en ESTUDIO-CAPAS-Y-ARQUITECTURA.md.)

Controller

Qué es: la puerta de entrada. Recibe el pedido HTTP, lo traduce a objetos Java, llama al service y devuelve la respuesta.

Lo importante: en Bajoneá el **controller es tonto a propósito**. Cero lógica de negocio.

Cómo contarlo: *"Mis controllers no deciden nada. Reciben, delegan y devuelven. Toda la lógica está en los services, así que si mañana quiero exponer lo mismo por otro medio, no toco nada de la lógica."*

Service

Qué es: la capa donde vive la lógica de negocio. Valida reglas, orquesta repositories, convierte entidades en DTOs y coordina con otros services.

En Bajoneá: 18 services. Los más grandes son `AuthService` (447 líneas) y `ProductoService` (440).

Ejemplo concreto de "lógica de negocio": *"no se puede agregar al carrito un producto de otro comercio"*. Eso no es HTTP ni base de datos: es una regla del negocio, y por eso vive en `CarritoService`.

Repository / Repository Pattern

Qué es: la capa que habla con la base de datos. El patrón consiste en **esconder detrás de una interfaz** cómo se guardan y se traen los datos.

Por qué es un patrón y no solo "el que hace las consultas": porque el service pide `productoRepository.findById(5)` sin saber si detrás hay MySQL, PostgreSQL, o un archivo. **El service no sabe SQL.**

En Bajoneá: 26 repositories, todos **interfaces** que extienden `JpaRepository`. Spring Data genera la implementación sola en tiempo de ejecución.

La regla del proyecto: los repositories son "tontos" — solo queries, nunca lógica.

DTO (Data Transfer Object)

Qué es: un objeto que existe **solo para llevar datos** entre capas, sin lógica adentro.

En Bajoneá: 30 de request (lo que entra) y 32 de response (lo que sale).

Las 5 razones de por qué no se devuelven las entidades

1. **Seguridad.** `Usuario` tiene `passwordHash` . Devolver la entidad mandaría el hash al navegador.
2. **Distinto público, distinto detalle.** El mismo comercio tiene 3 DTOs: público (sin el DNI del representante), propio, y de administrador.
3. **Desacoplar la API de la base.** Renombrás una columna y el frontend ni se entera.
4. **Los datos no coinciden 1 a 1.** `ProductoResponseDTO` lleva `nombreCategoria` y `nombreComercio` , que están en otras tablas.
5. **Evitar la serialización infinita.** Si `Comercio` tiene productos y cada producto apunta de vuelta al comercio, convertir eso a JSON entra en un bucle.

El mejor ejemplo para la mesa: los tres DTOs de comercio.

Entity

Qué es: una clase Java que representa una tabla. Cada atributo es una columna, cada objeto es una fila.

En Bajoneá: 26 entidades + 1 clase de apoyo (`ProductoTagId`).

Las convenciones del proyecto: `@Getter` de clase, `@Setter` campo por campo (**nunca en el `id`**), `@EqualsAndHashCode(of = "id")` , `LAZY` en toda relación, y **cero comentarios adentro**.

Por qué nunca `@Data` de Lombok: porque genera `equals` / `hashCode` sobre todos los campos, y en relaciones bidireccionales eso provoca **recursión infinita**.

ORM (Object-Relational Mapping)

Qué es: la herramienta que traduce entre el mundo de objetos de Java y el mundo de tablas de SQL.

El problema que resuelve: en Java tenés objetos con referencias (`producto.getComercio()`); en SQL tenés tablas con claves foráneas y `JOIN` . Se llama *impedance mismatch* — no encajan naturalmente.

En Bajoneá: **Hibernate**, usado a través de **JPA**.

Lo concreto: escribís `productoRepository.save(producto)` y Hibernate genera el `INSERT` . Nunca escribís SQL a mano (*salvo en dos `@Query` puntuales*).

Las ventajas: menos código, portable entre bases, y menos errores de tipeo en SQL.

Las desventajas (decilas vos, muestra criterio): el SQL generado no siempre es óptimo, aparecen problemas como el N+1, y hay que entender qué está haciendo por debajo o te sorprende.

JPA vs Hibernate — la distinción que conviene tener clara

	JPA	Hibernate
Qué es	La especificación (el estándar de Java)	Una implementación de esa especificación
Analogía	La norma que dice cómo debe ser un enchufe	El enchufe fabricado por una marca
En el código	<code>@Entity</code> , <code>@Id</code> , <code>@ManyToOne</code>	El motor que ejecuta eso

Por qué importa: si programás contra JPA, teóricamente podés cambiar Hibernate por otra implementación (EclipseLink) sin tocar el código.

BASE DE DATOS

Clave primaria (PK)

Qué es: la columna que identifica de forma única a cada fila.

En Bajoneá: casi todas son `Integer` autogenerado con `@GeneratedValue(strategy = IDENTITY)` — o sea, lo genera MySQL.

Las excepciones, que son buenos ejemplos:

- `Provincia` y `Localidad` usan `String` , con **códigos oficiales de la API Georef**, no autogenerados.
- `ProductoTag` usa una **clave compuesta** (`producto_id`, `tag_id`) .
- Las entidades de la cadena de identidad **comparten el id de su padre** vía `@MapsId` .

Clave foránea (FK)

Qué es: una columna que apunta a la clave primaria de otra tabla. Es lo que materializa una relación.

En Bajoneá: se declara con `@JoinColumn(name = "comercio_id")`.

La regla de dónde va: en una relación **uno a muchos**, la FK va del lado del **muchos**. La tabla `producto` tiene `comercio_id`, no al revés — porque una columna guarda un solo valor.

Lo que garantiza: que no puedas insertar un producto con un `comercio_id` que no existe. Es **integridad referencial**, y la hace cumplir la base, no la aplicación.

Relaciones (cardinalidad)

Tipo	Anotación JPA	Ejemplo en Bajoneá
Uno a uno	@OneToOne	Un cliente tiene un carrito.
Uno a muchos	@OneToMany	Un producto tiene muchas imágenes.
Muchos a uno	@ManyToOne	Muchos productos pertenecen a un comercio.
Muchos a muchos	Tabla intermedia	Un producto tiene muchos tags, un tag está en muchos productos.

Cómo se implementa la N:M: con una tabla intermedia. En Bajoneá es `ProductoTag`, cuya clave primaria es **el par completo** (`producto_id`, `tag_id`).

Por qué hace falta una tabla intermedia: porque una columna guarda un solo valor. No podés poner "los tags 2, 5 y 7" en una celda sin violar la primera forma normal.

mappedBy

Qué es: en una relación bidireccional, le dice a JPA **de qué lado vive la FK**.

```
@OneToMany(mappedBy = "producto")
private List<ImagenProducto> imagenes;
```

Traducido: *"esta relación ya está mapeada del otro lado, en el campo `producto` de `ImagenProducto`".* O sea: la FK está en `imagen_producto`.

Sin `mappedBy`, Hibernate crearía una tercera tabla intermedia innecesaria.

LAZY vs EAGER

	LAZY (lo que usa Bajoneá)	EAGER
Cuándo carga la relación	Solo si la pedís	Siempre, junto con la entidad
Ejemplo	Traés un producto; su comercio no se consulta hasta hacer <code>getComercio()</code>	Traés un producto y viene el comercio también

Por qué LAZY en todo: si fuera EAGER, traer un producto traería su comercio, y el comercio su dueño, y el dueño su persona jurídica... una consulta arrastraría media base.

El precio de LAZY:

- El problema N+1 (abajo).
- `LazyInitializationException` si intentás navegar la relación fuera de la transacción.

(En `GeografiaService` hay un ejemplo concreto de cómo se evita ese segundo problema: reutilizando el `provinciaId` que ya venía en el parámetro en vez de hacer `Localidad.getProvincia().getId()`.)

El problema N+1

Qué es: cuando recorrés una lista de N objetos y para cada uno pedís una relación, hacés **1 consulta + N consultas**.

Ejemplo concreto: traés 100 productos (1 consulta) y para cada uno pedís su comercio para mostrar el nombre (100 consultas). Total: **101 viajes a la base**.

Cómo se resuelve: con `JOIN FETCH` en la consulta, para traer todo de una.

Dónde aparece en Bajoneá — decilo antes de que te lo marquen: `AdministradorService.adminResponseDTO` hace varias consultas por cada comercio del listado (dirección, horarios, redes). Con pocos comercios no se nota, pero **es un N+1 real**.

Transacción y ACID

Qué es: un conjunto de operaciones que se ejecutan **como una sola unidad**: o se hacen todas, o no se hace ninguna.

ACID son las 4 propiedades:

Letra	Qué significa
Atomicidad	Todo o nada.
Consistencia	La base queda siempre en un estado válido.
Islamiento	Dos transacciones simultáneas no se pisan.
Durabilidad	Una vez confirmada, queda guardada aunque se corte la luz.

En Bajoneá: `@Transactional` sobre los services. Ejemplo: `confirmarPedido` crea el pedido, sus detalles, vacía el carrito y crea la notificación. Si algo falla, **se deshace todo**.

Bloqueo pesimista vs optimista

	Pesimista (lo que usa Bajoneá)	Optimista
Cómo funciona	Bloquea la fila desde el principio	Deja pasar a todos y detecta el choque al guardar
SQL	<code>SELECT ... FOR UPDATE</code>	Una columna <code>version</code>
Cuándo conviene	Cuando el conflicto es probable	Cuando es raro
Costo	Los demás esperan	Reintentar

En Bajoneá: `@Lock(LockModeType.PESSIMISTIC_WRITE)` en dos métodos de `UsuarioRepository` .

El problema que resuelve — "lost update":

```
Request A: lee intentosFallidos = 2
Request B: lee intentosFallidos = 2      (simultáneo)
Request A: escribe 3
Request B: escribe 3                    (pisa lo de A)
Resultado: 3 en vez de 4 → la cuenta nunca se bloquea
```

Baja lógica (soft delete)

Qué es: en vez de borrar la fila, marcarla como inactiva.

En Bajoneá: categorías y tags usan `activo = false` + `fechaBaja` . Las redes sociales usan `fechaBaja IS NULL` para saber si están vigentes.

Por qué: si borrás una categoría con 40 productos, esos productos quedan huérfanos. Con baja lógica los datos históricos siguen coherentes, **y se puede revertir**.

Migración de base de datos / Flyway

Qué es: versionar los cambios de esquema como archivos SQL numerados, para que todos los entornos tengan la misma estructura.

Cómo funciona: cada archivo (`v1__...sql` , `v2__...sql`) se ejecuta una sola vez, y Flyway lleva el registro en una tabla `flyway_schema_history` .

En Bajoneá: el esquema lo gestiona **exclusivamente Flyway**, con `spring.jpa.hibernate.ddl-auto=validate` .

Ese `validate` es importante: significa que **Hibernate NO modifica la base**, solo verifica al arrancar que las entidades coincidan con el esquema. Si no coinciden, **la aplicación no arranca**.

Por qué nunca `update` : porque le dejarías a Hibernate decidir cómo modificar la base, sin control, sin revisión y sin forma de replicarlo en otro entorno. Es una de las reglas no negociables del proyecto.

ETL (Extract, Transform, Load)

Qué es: un proceso que **extrae** datos de una fuente, los **transforma** al formato que necesitás, y los **carga** en tu base.

En Bajoneá: un script Node.js (`etl-georef.mjs`) que consultó la **API Georef** del gobierno argentino y cargó **24 provincias y ~4038 localidades**.

El detalle de buena práctica: el script es **idempotente** — usa `ON DUPLICATE KEY UPDATE` , así que podés correrlo dos veces y obtenés los mismos conteos, sin duplicados. **Se verificó corriendolo dos veces**.

SEGURIDAD

Autenticación vs Autorización

	Autenticación	Autorización
La pregunta	"¿Quién sos?"	"¿Qué podés hacer?"
En Bajoneá	Validar email + contraseña, emitir el JWT	Las reglas <code>hasRole(...)</code> de <code>SecurityConfig</code>
Error asociado	401 Unauthorized	403 Forbidden

Truco para no confundir el 401: está mal nombrado en el estándar HTTP. Dice "unauthorized" pero significa "**unauthenticated**". El 403 es el verdadero "no autorizado".

JWT (JSON Web Token)

Qué es: un token con tres partes separadas por puntos: `cabecera.contenido.firma`. El contenido son datos ("claims") que el servidor mete, y la firma garantiza que **nadie los modificó**.

Lo más importante, y es pregunta segura de mesa:

El contenido de un JWT **no está encriptado**, está apenas codificado en Base64. Cualquiera lo puede leer pegándolo en `jwt.io`. **Lo que protege el JWT no es el secreto del contenido, es la integridad**: si alguien le cambia el rol de CLIENTE a ADMINISTRADOR, la firma deja de coincidir y se rechaza. **Por eso nunca se mete una contraseña ni un dato sensible adentro**.

Los claims de Bajoneá: `sub` (email), `userId`, `rol`, **`sessionId`**, `iat`, `exp` (24 horas).

Stateless

Qué significa: que el servidor **no guarda estado** entre requests. Cada pedido llega con toda la información necesaria.

La ventaja: podés tener 10 servidores y no importa a cuál te toque, porque ninguno guarda nada tuyo.

En Bajoneá hay un matiz que hay que saber explicar:

Spring Security está configurado como `STATELESS` — no crea `HttpSession` ni cookie de servlet. Pero **sí tengo una tabla `sesion` propia**, que el filtro consulta en cada request.

No es una contradicción: son dos cosas distintas. `STATELESS` habla de la sesión de servlet; mi tabla `sesion` es estado de negocio, a nivel aplicación. Lo hice porque un JWT puramente stateless **no se puede revocar**, y yo necesitaba poder cerrar sesiones de verdad.

Hash y BCrypt

Qué es un hash: una función de **una sola dirección**. De la contraseña sacás el hash, pero del hash **no podés volver** a la contraseña.

Cómo funciona el login entonces: no se "desencripta" nada. Se hashea lo que el usuario escribió y se compara con el hash guardado.

Por qué BCrypt y no SHA-256:

	SHA-256	BCrypt
Velocidad	Muy rápido	Lento a propósito
Salt	Hay que agregarlo a mano	Automático, uno por contraseña
Para contraseñas	Malo	Bueno

Lo que parece contradictorio pero no lo es: que sea lento es la ventaja. Si un atacante roba la base, con SHA-256 podría probar millones de contraseñas por segundo; con BCrypt, unas pocas miles. La demora de milisegundos en un login legítimo es imperceptible; en un ataque de fuerza bruta, es la diferencia entre horas y siglos.

El salt es un valor aleatorio que se le agrega a cada contraseña antes de hashear. Hace que dos usuarios con la misma contraseña tengan hashes **distintos**, y anula las "rainbow tables" (diccionarios precalculados de hashes).

CORS (Cross-Origin Resource Sharing)

Qué es: una protección del navegador que impide que una página de un origen (dominio + puerto) llame a otro origen, salvo que ese otro lo permita explícitamente.

Por qué existe: para que un sitio malicioso no pueda hacer requests a tu banco usando tu sesión.

En Bajoneá: el frontend está en `localhost:5501` y el backend en `localhost:8080` — **distinto puerto = distinto origen**. Sin configurar CORS, el navegador bloquearía todas las llamadas.

La configuración actual permite todos los orígenes (`*`), que es cómodo en desarrollo. *(En producción convendría restringirlo al dominio propio — buen punto para mencionar si preguntan.)*

CSRF (Cross-Site Request Forgery)

Qué es: un ataque donde un sitio malicioso hace que tu navegador mande un request a otro sitio donde estás logueado, aprovechando que **las cookies se mandan solas**.

Por qué Bajoneá lo puede desactivar:

Porque **no uso cookies de sesión**. Mi token va en el header `Authorization` , y un sitio de terceros **no puede leerlo ni agregarlo** a una petición. El ataque CSRF depende del envío automático de cookies; sin cookies, no aplica.

Enumeración de usuarios

Qué es: una filtración de información donde el sistema responde **distinto** según exista o no una cuenta, permitiendo armar una lista de usuarios registrados.

Cómo se evita en Bajoneá:

- El login devuelve el mismo mensaje si el email no existe o si la contraseña está mal.
- Recuperación, reactivación y reenvío devuelven siempre *"Si existe una cuenta asociada a ese email..."*, **sin importar si existe**.

El bug real que se corrigió: esos endpoints lanzaban una excepción que llegaba al handler y devolvía un **404 real**, contradiciendo el mensaje ambiguo. Se corrigió con `ifPresent` .

IDOR (Insecure Direct Object Reference)

Qué es: cuando un usuario puede acceder a datos ajenos **cambiando un id en la URL**.

Cómo se evita en Bajoneá — dos mecanismos:

1. El id no viene por URL, sale del token. Los endpoints son `/clientes/perfil` , no `/clientes/{id}` . Con `@AuthenticationPrincipal` el service saca el `userId` del JWT, que está firmado y no se puede falsear.

2. Cuando sí hay un id en la URL, se valida el dueño, y se devuelve 404, no 403:

```
if (!producto.getComercio().getId().equals(comercio.getId())) {
    throw new RecursoNoEncontradoException("Producto no encontrado");
}
```

Por qué 404 y no 403: un 403 confirmaría que ese producto existe pero no es tuyo. Probando ids podrías mapear el sistema. Con 404 uniforme, **no podés distinguir "no existe" de "no es tuyo"**.

Rate limiting

Qué es: limitar cuántas veces se puede llamar a un endpoint en un período.

En Bajoneá: `RateLimitFotoRegistroFilter` limita a **5 firmas de Cloudinary por minuto por IP**, solo en los dos endpoints de firma del registro.

Por qué justo esos dos: porque son los **únicos públicos sin login** de firma. Todos los demás piden token, así que sabés quién es. Sin freno, alguien podría hacer un script que pida firmas infinitas y llene (y facture) la cuenta de Cloudinary.

El algoritmo es "ventana fija": un contador por IP que se resetea cada minuto.

Las limitaciones, dichas de frente: está en memoria (se pierde al reiniciar), sirve para un solo servidor, y tiene el borde de la ventana fija (5 en el segundo 59 y 5 más en el 61).

SPRING Y JAVA

Spring Boot

Qué es: un framework de Java que arma aplicaciones web con configuración mínima.

Lo que aporta: trae un servidor web embebido (Tomcat), autoconfigura las dependencias que detecta, y te evita cientos de líneas de configuración XML.

En Bajoneá: Spring Boot 3.x sobre Java 21, con Maven.

Inyección de dependencias (DI) e IoC

Qué es: en vez de que una clase cree lo que necesita (`new ProductoRepository()`), **se lo pide a alguien que se lo da ya armado**.

IoC (Inversión de Control) es el nombre del principio: quien controla la creación de los objetos no es tu clase, es el framework.

En Bajoneá:


```
@Service
@RequiredArgsConstructor
public class ProductoService {
    private final ProductoRepository productoRepository;    // Spring lo inyecta solo
}
```

`@RequiredArgsConstructor` de Lombok genera el constructor con los campos `final` , y Spring detecta ese constructor y le pasa los objetos.

Por qué es mejor que hacer `new` : porque la clase no depende de **cómo** se construye lo que usa. Podés cambiar la implementación sin tocarla, y en un test podés inyectarle un objeto falso.

Bean

Qué es: un objeto que Spring crea y administra. Se declaran con `@Component` , `@Service` , `@Repository` , `@Controller` , `@Configuration` , o con `@Bean` dentro de una clase de configuración.

En Bajoneá: los services, controllers, repositories, filtros, y los objetos de configuración (`Cloudinary` , `Resend` , `PasswordEncoder` , `OpenAPI`).

Por defecto son singletons: existe **una sola instancia** de cada uno en toda la aplicación, y todos comparten la misma.

Anotaciones (annotations)

Qué son: metadatos que se ponen sobre una clase, método o campo, y que el framework lee para saber qué hacer.

Las que más aparecen en Bajoneá:

Anotación	Qué hace
<code>@RestController</code>	Esta clase atiende HTTP y devuelve JSON.
<code>@Service</code>	Esta clase es un service; Spring la administra.
<code>@Entity</code>	Esta clase es una tabla.
<code>@Transactional</code>	Este método es una transacción.
<code>@Valid</code>	Validá el objeto antes de entrar al método.
<code>@Autowired</code> / <code>@RequiredArgsConstructor</code>	Inyectame las dependencias.

Cómo funcionan por debajo: con **reflexión** — el framework inspecciona las clases en tiempo de ejecución, ve las anotaciones y actúa. Por eso `@Retention(RUNTIME)` es obligatorio en las anotaciones custom.

Bean Validation

Qué es: el estándar de Java (`jakarta.validation`) para validar objetos con anotaciones.

En Bajoneá: las estándar (`@NotBlank` , `@Email` , `@Size` , `@Pattern` ...) más **12 anotaciones custom** (`@ValidarCuit` , `@ValidarTelefonoArgentino` , etc.).

Cómo se dispara: con `@Valid` sobre el `@RequestBody` del controller. Si algo falla, Spring lanza `MethodArgumentNotValidException` **antes** de entrar al método, y el handler la traduce a un 400 con el detalle por campo.

Sus límites (importante saberlos):

- Valida **un campo a la vez** — no puede expresar "obligatorio solo si otro campo vale X".
- No puede consultar la base** — no sabe si un email ya existe.

Por eso hay validaciones que necesariamente viven en el service.

Lombok

Qué es: una librería que genera código repetitivo en tiempo de compilación.

Anotación	Qué genera
<code>@Getter</code> / <code>@Setter</code>	Los <code>getX()</code> / <code>setX()</code> .
<code>@NoArgsConstructor</code>	Constructor vacío.
<code>@AllArgsConstructor</code>	Constructor con todos los campos.
<code>@RequiredArgsConstructor</code>	Constructor con los campos <code>final</code> .
<code>@Builder</code>	El patrón builder.
<code>@EqualsAndHashCode(of = "id")</code>	<code>equals</code> y <code>hashCode</code> usando solo el id.

Patrón Builder

Qué es: una forma de construir objetos con muchos campos, de manera legible.

```
Producto producto = Producto.builder()
    .comercio(comercio)
    .nombre("Pizza Muzzarella")
    .precio(new BigDecimal("8500"))
    .estado(EstadoProducto.DISPONIBLE)
    .build();
```

Por qué es mejor que un constructor de 8 parámetros: porque leyendo el código sabés qué es cada valor. Con `new Producto(comercio, categoria, "Pizza", null, precio, ...)` no sabés qué es el `null` .

En Bajoneá: todas las entidades tienen `@Builder` .

Optional<T>

Qué es: una forma explícita de decir "*esto puede venir vacío*", obligando a manejar ese caso.

```
Usuario usuario = usuarioRepository.findByEmail(email)
    .orElseThrow(() -> new RecursoNoEncontradoException("Usuario no encontrado"));
```

Por qué es mejor que devolver `null` : porque olvidarse de chequear un `null` es un `NullPointerException` esperando a pasar. Con `optional` el compilador te obliga a decidir qué hacer si está vacío.

Los métodos que se usan en Bajoneá: `orElseThrow(...)` , `orElse(null)` , `ifPresent(...)` , `map(...)` , `orElseGet(...)` .

Excepciones checked vs unchecked

	Checked (extends Exception)	Unchecked (extends RuntimeException)
¿Declararla?	Sí, con <code>throws</code>	No
¿Atraparla?	Obligatorio	No
Ejemplo	<code>IOException</code>	<code>NullPointerException</code>

Las 4 excepciones de Bajoneá son unchecked. Dos motivos:

1. No ensucian las firmas con `throws` .
2. Spring solo hace rollback automático con excepciones unchecked.

@RestControllerAdvice

Qué es: una clase que escucha las excepciones de todos los controllers y las traduce a respuestas HTTP.

En Bajoneá: `GlobalExceptionHandler` , con 8 handlers: las 4 excepciones propias + 4 de Spring.

Por qué es mejor que un `try/catch` en cada controller: porque el formato de error es idéntico en toda la API, y si querés cambiarlo tocás un solo archivo.

BigDecimal

Qué es: el tipo de Java para números decimales **exactos**.

Por qué no `double` para plata — pregunta clásica:

`double` usa punto flotante binario y **no puede representar exactamente los decimales**. En Java, `0.1 + 0.2` da `0.30000000000000004` . Con dinero eso es inaceptable: los centavos se van corriendo. `BigDecimal` guarda el número en decimal exacto. **Para plata, siempre `BigDecimal`** .

En Bajoneá: `Producto.precio` , `Pedido.subtotal` , `Pedido.total` , `DetallePedido.precioUnitario` , todos `BigDecimal(10,2)` .

Enum

Qué es: un tipo con una lista **cerrada** de valores posibles.

Por qué **STRING** y no **ORDINAL**:

ORDINAL guarda la **posición** (0, 1, 2). Si mañana insertás un valor **en el medio** del enum, **todas las filas guardadas cambian de significado en silencio**: un pedido que decía `ENTREGADO` pasaría a decir `RECHAZADO` sin que nadie toque la base. Con **STRING** el texto siempre significa lo mismo, y además podés leer la tabla sin traducir números.

Dato que sorprende: un enum de Java **puede tener atributos y métodos**. `MotivoRechazo` tiene un campo `etiqueta` con el texto legible ("Alto volumen de pedidos") y un `getEtiqueta()` .

HERRAMIENTAS Y SERVICIOS

Maven

Qué es: la herramienta que gestiona las dependencias y compila el proyecto.

Cómo funciona: declarás las librerías en `pom.xml` y Maven las descarga con todas sus dependencias transitivas.

Swagger / OpenAPI

Qué es: **OpenAPI** es el estándar para describir una API REST. **Swagger UI** es la página web que genera esa descripción de forma navegable e interactiva.

En Bajoneá: `springdoc-openapi` lo genera **automáticamente** leyendo los controllers, y `OpenApiConfig` agrega el esquema Bearer/JWT — que es lo que hace aparecer el botón **"Authorize"** para pegar el token una vez y probar todos los endpoints.

La ventaja de que sea automático: la documentación **no se desactualiza**, porque sale del código real.

Postman y Newman

Qué son: Postman es la herramienta para probar APIs manualmente. **Newman** es el que corre esas mismas colecciones **desde la línea de comandos**, para poder automatizarlas.

En Bajoneá: una colección de **203 requests en 20 carpetas**, con **416 assertions**, cubriendo los 69 endpoints reales del backend.

Playwright

Qué es: una herramienta de testing **end-to-end**: abre un navegador de verdad y simula lo que haría un usuario — hacer clic, escribir, navegar.

En Bajoneá: 9 specs, 36 tests. La última corrida verificada dio **35 en verde y 1 fallo conocido**, que es un bug de layout menor del panel de administrador (un botón flotante tapa el de editar cuando la lista es larga), **aceptado explícitamente como deuda de UI, no como falla de testing**.

Un detalle real muy contable: los tests suben **imágenes reales a Cloudinary**, no un mock. Fue una decisión explícita para probar el flujo completo de verdad.

Cloudinary

Qué es: un servicio en la nube para guardar y servir imágenes.

Cómo se usa en Bajoneá — el punto clave:

El **backend nunca recibe el archivo**. El frontend le pide una firma al backend, sube la imagen **directo a Cloudinary** con esa firma, y después le manda al backend solo la URL. El servidor no gasta ancho de banda ni disco.

La firma está calculada con el `apiSecret` , que **nunca sale del servidor**, y está atada a una **carpeta específica** derivada del id que sale del JWT. Por eso un comercio no puede subir a la carpeta de otro.

Resend

Qué es: un servicio para mandar emails transaccionales por **API REST**.

Por qué no SMTP: el proyecto usaba Brevo por SMTP, pero la cuenta nunca se activó (error `502 SMTP account is not yet activated`) y no hubo resolución con soporte. Se migró a Resend.

La diferencia técnica: SMTP es un protocolo de correo donde tu servidor "habla" ese idioma; una API REST es simplemente una llamada HTTP diciendo "mandá este mail". Más simple y más confiable.

Polling vs WebSockets

	Polling (lo que usa Bajoneá)	WebSockets
Cómo funciona	El cliente pregunta cada X segundos	El servidor avisa cuando pasa algo
Latencia	Hasta X segundos	Instantáneo
Complejidad	Muy baja	Alta
Carga	Una consulta por usuario cada X	Una conexión abierta por usuario

En Bajoneá: polling **cada 15 segundos** al contador de notificaciones no leídas.

Cómo justificarlo: *"Para esta escala alcanza. 15 segundos de demora en enterarte de un pedido no rompe nada, y WebSockets agregaría manejo de conexiones y reconexiones que no hacía falta. Lo que sí optimicé es que el endpoint del polling devuelva solo un número, no la lista completa."*

CONCEPTOS DE DISEÑO

Multi-tenancy / aislamiento por dueño

Qué es: que varios usuarios usen el mismo sistema **sin ver los datos de los demás**.

En Bajoneá — dos mecanismos:

- El id sale del **JWT**, no de la URL.
- Cuando hay un id en la URL, se valida el dueño y se devuelve **404**, no 403.

Idempotencia

Qué es: que hacer la misma operación una o diez veces dé el mismo resultado.

Ejemplos:

- GET es idempotente:** consultar mil veces no cambia nada.
- POST no lo es:** crear un producto dos veces crea dos productos.

En Bajoneá:

- El script ETL de Georef es idempotente (`ON DUPLICATE KEY UPDATE`).
- `aceptarPedido` **se hace idempotente a la fuerza** con la validación de estado: el segundo intento devuelve 409 en vez de duplicar el efecto. **Eso es lo que protege contra el doble clic.**

Snapshot de datos

Qué es: copiar un valor en el momento de una operación, en vez de referenciarlo.

En Bajoneá: `DetallePedido.precioUnitario` copia el precio del producto y lo congela (`updatable = false`).

Por qué: *"Un pedido es un documento histórico. Si el detalle apuntara al precio actual del producto, tu historial de compras cambiaría solo cada vez que el comercio ajusta precios."*

El contraste que lo explica todo: `ItemCarrito` **no** guarda el precio. El carrito es algo **vivo** (querés ver el precio de hoy); el pedido es algo **cerrado** (querés ver el de cuando compraste).

Referencia polimórfica

Qué es: un par de columnas (`tipo` + `id`) que puede apuntar a **cualquier** tabla.

En Bajoneá: `Notificacion.entidadTipo` + `entidadId` .

El **trade-off**: ganás flexibilidad (una tabla sirve para todo), **pero la base no puede poner una FK** — no sabe a qué tabla apunta. La integridad queda a cargo de la aplicación.

Defensa en profundidad

Qué es: poner **varias capas** de protección para la misma regla, de modo que si una falla, otra la cubre.

El mejor ejemplo de Bajoneá — el email único:

- | | |
|---|-------------------------------|
| 1. <code>@NotBlank</code> + <code>@ValidarFormatoEmail</code> en el DTO | → valida el formato |
| 2. <code>existsByEmail()</code> en el service | → valida que no exista |
| 3. <code>UNIQUE</code> en la columna de la base | → garantía final ante carrera |

Por qué hacen falta las tres: entre el paso 2 y el `INSERT` puede meterse otro request (una condición de carrera **check-then-act**). La única garantía real es el **UNIQUE** de la base. El paso 2 existe para dar un mensaje claro en el 99,9% de los casos, no para garantizar nada.

DRY (Don't Repeat Yourself)

Qué es: no duplicar la misma lógica en varios lugares.

Ejemplos en Bajoneá:

- `ComercioValidaciones` — la regla de "al menos delivery o retiro", que usan **dos** services.
- `NotificacionService.crear` — antes copiado en tres services, se centralizó.
- `renderTopBar` , `renderPedidoEstadoHeader` — componentes visuales compartidos.

El contraejemplo consciente: `aTitleCase` **está duplicado** en Java y en JavaScript, **a propósito**, para que backend y frontend capitalicen exactamente igual. A veces duplicar es más simple que armar infraestructura para compartir.

Fail-safe vs fail-open

Fail-safe (o fail-closed): ante la duda, denegar. **Fail-open**: ante la duda, permitir.

En Bajoneá:

- Fail-safe**: un comercio sin horarios cargados se considera **cerrado**. Mejor no aceptar un pedido que no se va a poder cumplir.
- Fail-open**: el `DireccionExclusionMutuaValidator` , si el DTO no declara los campos que espera, **no falla**. El criterio es que ahí el problema sería de configuración del DTO, no un dato inválido del usuario.

En seguridad siempre se prefiere fail-safe.

Índice del glosario

Arquitectura y capas

API REST · Arquitectura en capas · Controller · Service · Repository Pattern · DTO · Entity · ORM · JPA vs Hibernate

Base de datos

Clave primaria · Clave foránea · Relaciones · `mappedBy` · LAZY vs EAGER · Problema N+1 · Transacción y ACID · Bloqueo pesimista vs optimista · Baja lógica · Flyway · ETL

Seguridad

Autenticación vs Autorización · JWT · Stateless · Hash y BCrypt · CORS · CSRF · Enumeración de usuarios · IDOR · Rate limiting

Spring y Java

Spring Boot · Inyección de dependencias e IoC · Bean · Anotaciones · Bean Validation · Lombok · Patrón Builder · `Optional<T>` · Excepciones checked vs unchecked · `@RestControllerAdvice` · `BigDecimal` · Enum

Herramientas

Maven · Swagger/OpenAPI · Postman y Newman · Playwright · Cloudinary · Resend · Polling vs WebSockets

Conceptos de diseño

Multi-tenancy · Idempotencia · Snapshot de datos · Referencia polimórfica · Defensa en profundidad · DRY · Fail-safe vs fail-open